

LAPORAN PRAKTIKUM
STRUKTUR DATA
“ SLL – SINGLE LINKED LIST”



disusun Oleh:
FILZI JELILA INDA ROBBANI
(2511533019)

Dosen Pengampu:
Dr. WAHYUDI, S.T, M.T
Asisten Praktikum:
ZAHRAN AZARIA ANVAYA

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadiran Allah SWT, atas limpahan rahmat dan karunia-Nya sehingga praktikum Struktur Data kali ini dapat saya selesaikan dengan baik dan lancar. Shalawat serta salam semoga senantiasa tercurahkan kepada junjungan kita Nabi Muhammad SAW.

Laporan ini saya susun untuk memenuhi salah satu tugas pada mata kuliah Praktikum Struktur Data di Universitas Andalas. Laporan ini diharapkan dapat menambah wawasan mengenai penggunaan struktur data Single Linked List pada ADT .

Saya menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum yang telah membimbing serta memberikan arahan selama proses pembelajaran di kelas maupun di laboratorium komputer. Selain itu, saya juga berterima kasih kepada teman-teman praktikan atas bantuan dan dukungan yang diberikan, sehingga laporan ini dapat terselesaikan dengan baik.

Saya menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, saya mohon maaf apabila terdapat kekurangan maupun kesalahan dalam penyusunan laporan ini. Harapan saya, semoga laporan ini dapat memberikan manfaat serta menambah pengetahuan bagi pembaca sekalian.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan.....	5
1.3 Manfaat	5
BAB II PEMBAHASAN	6
2.1 Langkah Kerja Praktikum	6
2.1.1 Membuat kelas pertama (NodeSLL).....	6
2.1.2 Membuat kelas kedua (TambahSLL).....	6
2.1.3 Membuat program ketiga (PencarianSLL).....	10
2.1.4 Membuat program keempat (HapusSLL)	11
2.2 Tugas Praktikum	15
2.2.1 Deskripsi Tugas	15
2.2.2 Kelas ADT	15
2.2.3 Kelas Driver	17
BAB III PENUTUP.....	12
3.1 Kesimpulan	12
DAFTAR PUSTAKA.....	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya, telah dipelajari mengenai struktur data Queue yang digunakan untuk mengelola data secara terurut dengan prinsip FIFO (First In First Out). Queue membantu mahasiswa memahami bagaimana data diproses sesuai urutan masuk dan keluar, serta bagaimana pengelolaan data dilakukan secara terstruktur dalam pemrograman Java. Selain itu, praktikum tersebut juga memberikan pemahaman mengenai penggunaan array dalam implementasi struktur data sederhana.

Selanjutnya, pada praktikum kali ini dilakukan pengembangan materi dengan mempelajari Single Linked List (SLL) menggunakan bahasa pemrograman Java. Single Linked List merupakan struktur data linear yang terdiri dari kumpulan node yang saling terhubung melalui pointer. Setiap node memiliki dua bagian utama, yaitu data dan alamat menuju node berikutnya. Berbeda dengan array yang memiliki ukuran tetap, Single Linked List bersifat dinamis sehingga penambahan maupun penghapusan data dapat dilakukan dengan lebih fleksibel.

Dalam praktikum ini dilakukan beberapa operasi dasar pada Single Linked List, seperti traversal atau penelusuran data, pencarian data, penambahan node di awal, tengah, dan akhir list, serta penghapusan node pada posisi tertentu. Melalui implementasi tersebut, mahasiswa dapat memahami cara kerja pointer dan hubungan antar node dalam struktur data dinamis.

Dengan demikian, praktikum ini menjadi lanjutan yang penting untuk memperkuat pemahaman mahasiswa mengenai struktur data linear, khususnya dalam pengelolaan data yang lebih fleksibel, dinamis, dan efisien menggunakan konsep linked list berbasis objek.

1.2 Tujuan

Tujuan dilakukannya praktikum ini adalah sebagai berikut:

- a. Memahami konsep dasar Single Linked List dalam Java.
- b. Memahami penggunaan node dan pointer pada linked list.
- c. Membuat program penelusuran data pada linked list.
- d. Membuat program penambahan node pada linked list.
- e. Membuat program penghapusan node pada linked list.
- f. Mengimplementasikan Single Linked List menggunakan Java..
- g. Melatih kemampuan dalam mengimplementasikan konsep teori struktur data ke dalam program Java.

1.3 Manfaat

- a. Mahasiswa dapat memahami penggunaan struktur data dinamis dalam Java.
- b. Mahasiswa dapat memahami bagaimana hubungan antar node pada linked list.
- c. Meningkatkan kemampuan dalam mengelola data secara efisien dan fleksibel.
- d. Menambah pemahaman mengenai operasi dasar linked list seperti traversal, insertion, searching, dan deletion.
- e. Meningkatkan kemampuan problem solving dalam pemrograman.

BAB II

PEMBAHASAN

2.1 Langkah Kerja Praktikum

2.1.1 Membuat kelas pertama (NodeSLL)

- 1) Pertama buka apl Eclipse IDE for Java Developers, lalu buatlah package baru dengan nama pekan5_nim terus pilih “New”, pilih class. Buat nama class “NodeSLL_2511533019”.
- 2) Lalu, masukkan syntax seperti berikut.

```
1) package Pekan5_2511533019;
2)
3) public class NodeSLL_2511533019 {
4)     //node bagian data
5)     int data_3019;
6)     //pointer ke node berikutnya
7)     NodeSLL_2511533019 next_3019;
8)     //konstruktor menginisialisasi node dengan data
9)     public NodeSLL_2511533019(int data_3019) {
10)         this.data_3019= data_3019;
11)         this.next_3019 = next_3019;
12)     }
13)
14) }
```

Program pertama yaitu NodeSLL_2511533019 digunakan sebagai dasar pembentukan node pada linked list. Pada kelas ini terdapat atribut data_3019 yang berfungsi menyimpan data dan atribut next_3019 yang digunakan sebagai pointer menuju node berikutnya. Selain itu, terdapat konstruktor yang digunakan untuk menginisialisasi data ketika node baru dibuat. Kelas ini menjadi komponen utama karena seluruh operasi linked list bergantung pada pembentukan node tersebut.

2.1.2 Membuat kelas kedua (TambahSLL)

- 1) Klik kanan package pekan5 dan pilih “New”, pilih class. Lalu buat class baru dengan nama “TambahSLL_2511533019.”
- 2) Masukkan syntax seperti berikut.

```

1) package Pekan5_2511533019;
2)
3) public class TambahSLL_2511533019 {
4)     public static NodeSLL_2511533019
insertAtFront_3019(NodeSLL_2511533019 head_3019, int
value_3019) {
5)         NodeSLL_2511533019 new_node = new
NodeSLL_2511533019(value_3019);
6)         new_node.next_3019= head_3019;
7)         return new_node;
8)     }
9)     //fungsi menambahkan node diakhir SLL
10)    public static NodeSLL_2511533019
insertAtEnd_3019(NodeSLL_2511533019 head_3019, int
value_3019) {
11)        //buat sebuah node dengan sebuah nilai
12)        NodeSLL_2511533019 newNode_3019 = new
NodeSLL_2511533019(value_3019);
13)        // jika list kosong maka node jadi head
14)        if (head_3019 == null) {
15)            return newNode_3019;
16)        }
17)        //simpan head ke variabel sementara
18)        NodeSLL_2511533019 last = head_3019;
19)        // telusuri ke node akhir while {
20)            while (last.next_3019 != null) {
21)                last = last.next_3019;
22)            }
23)            //ubah pointer
24)            last.next_3019 = newNode_3019;
25)            return head_3019;
26)        }
27)        static NodeSLL_2511533019 GetNode_3019 (int
data_3019) {
28)            return new
NodeSLL_2511533019(data_3019) ;
29)        }
30)        static NodeSLL_2511533019
insertPos_3019(NodeSLL_2511533019 headNode_3019, int
posistion_3019, int value_3019) {
31)            NodeSLL_2511533019 head_3019=
headNode_3019;
32)            if(posistion_3019 < 1)
33)                System.out.print("invalid
position");
34)            if (posistion_3019 == 1) {
35)                NodeSLL_2511533019 new_node =
new NodeSLL_2511533019(value_3019);
36)                new_node.next_3019 = head_3019;
37)                return new_node;
38)            } else {
39)                while (posistion_3019-- != 0) {
40)                    if (posistion_3019 == 1) {
41)                        NodeSLL_2511533019
newNode_3019 = GetNode_3019(value_3019);
42)                        newNode_3019.next_3019 = headNode_3019.next_3019;

```

```

43)         headNode_3019.next_3019 = newNode_3019;
44)                                     break;
45)                                     }
46)         headNode_3019 =
headNode_3019.next_3019;
47)     }
48)     if (posistion_3019 != 1)
49)         System.out.print ("Posisi
di luar jangkauan");}
50)         return head_3019; }
51)     public static void printList_3019
(NodeSLL_2511533019 head_3019) {
52)         NodeSLL_2511533019 curr_3019 =
head_3019;
53)         while (curr_3019.next_3019 !=
null) {
54)             System.out.print
(curr_3019.data_3019 + "-->");
55)             curr_3019 =
curr_3019.next_3019;
56)         }
57)         if (curr_3019.next_3019== null)
{
58)             System.out.print
(curr_3019.data_3019) ;
59)             System.out.println();}
60)         }
61)
62)     public static void main (String[] args)
{
63)         // buat linked list 2->3-
>5->6
64)         NodeSLL_2511533019
head_3019 = new NodeSLL_2511533019(2);
65)         head_3019.next_3019 = new
NodeSLL_2511533019(3) ;
66)         head_3019.next_3019.next_3019 = new
NodeSLL_2511533019(5);
67)         head_3019.next_3019.next_3019.next_3019 = new
NodeSLL_2511533019(6);
68)
69)         // cetak list asli
70)         System.out.print("Senarai
berantai awal:");
71)         printList_3019(head_3019);
72)
73)         // tambahkan node baru di
depan
74)         System.out.print("tambah 1
simpul di depan: ");
75)         int data_3019 = 1;
76)         head_3019=
insertAtFront_3019 (head_3019, data_3019);
77)

```



```

78)                                     // cetak update list
79)                                     printList_3019(head_3019);
80)
81)                                     // tambahkan node baru di
    belakang
82)                                     System.out.print("tambah 1
    simpul di belakang: ");
83)                                     int data2 = 7;
84)                                     head_3019 =
    insertAtEnd_3019(head_3019, data2);
85)
86)                                     // cetak update list
87)                                     printList_3019(head_3019);
88)                                     System.out.print("tambah 1
    simpul ke data 4: ");
89)                                     int data3 = 4;
90)                                     int pos = 4;
91)                                     head_3019 =
    insertPos_3019(head_3019, pos, data3);
92)                                     // cetak update list
93)                                     printList_3019(head_3019);
94)                                     }
95)                                     }

```

Program kedua yaitu **TambahSLL_2511533019** berfungsi ntuk melakukan operasi penambahan node pada Single Linked List. Method `insertAtFront_3019()` digunakan untuk menambahkan node di bagian depan linked list dengan cara mengarahkan pointer node baru ke head lama kemudian menjadikan node baru sebagai head. Method `insertAtEnd_3019()` digunakan untuk menambahkan node di bagian akhir linked list dengan cara menelusuri hingga node terakhir lalu menghubungkan node terakhir tersebut dengan node baru. Selain itu, method `insertPos_3019()` digunakan untuk menyisipkan node pada posisi tertentu di dalam linked list. Pada program ini awalnya dibuat linked list berisi data 2, 3, 5, dan 6. Setelah itu dilakukan penambahan node di depan dengan data 1, penambahan node di belakang dengan data 7, dan penambahan node di tengah dengan data 4 sehingga menghasilkan linked list yang lebih lengkap dan teratur.

3) Run program maka akan tampil output seperti berikut.

```
Console X
<terminated> TambahSLL_2511533019 [Java Application] C:\Users\Filzi Jelik
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

2.1.3 Membuat program ketiga (PencarianSLL)

- 1) Klik kanan package pekan5 dan pilih “New”, pilih class. Lalu buat class baru dengan nama “PencarianSLL_2511533019.”
- 2) Lalu buat/masukkan syntax seperti berikut.

```
1) package Pekan5_2511533019;
2)
3) public class PencarianSLL_2511533019 {
4)     static boolean searchKey_3019(NodeSLL_2511533019
head_3019, int key_3019) {
5)         NodeSLL_2511533019 curr_3019 = head_3019;
6)         while (curr_3019 != null) {
7)             if (curr_3019.data_3019 ==
key_3019)
8)                 return true;
9)             curr_3019 =
curr_3019.next_3019; }
10)        return false; }
11)    public static void traversal_3019(NodeSLL_2511533019
head_3019) {
12)        //mulai dari head
13)        NodeSLL_2511533019 curr_3019 = head_3019;
14)        //telusuri sampai pointer null
15)        while (curr_3019 != null) {
16)            System.out.print(" " +
curr_3019.data_3019);
17)            curr_3019 = curr_3019.next_3019; }
18)        System.out.println(); }
19)    public static void main(String[] args) {
20)        // TODO Auto-generated method stub
21)        NodeSLL_2511533019 head_3019 = new
NodeSLL_2511533019(14);
22)        head_3019.next_3019 = new
NodeSLL_2511533019(21);
23)        head_3019.next_3019.next_3019 = new
NodeSLL_2511533019(13);
24)        head_3019.next_3019.next_3019.next_3019
= new NodeSLL_2511533019(30);
25)        head_3019.next_3019.next_3019.next_3019.next_3019 =
new NodeSLL_2511533019(10);
26)        System.out.print("Penelusuran SLL : ");
27)        traversal_3019(head_3019);
28)        //data yang akan dicari
29)        int key_3019= 30;
```

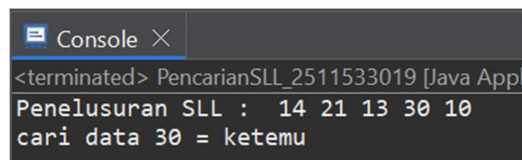
```

30)         System.out.print("cari data "+key_3019+
" = ");
31)         if (searchKey_3019(head_3019,
key_3019))
32)             System.out.println("ketemu");
33)         else
34)             System.out.println("tidak ada");
35)     }}

```

Program ketiga yaitu **PencarianSLL_2511533019** yang digunakan untuk melakukan traversal dan pencarian data pada Single Linked List. Pada class ini terdapat method `searchKey_3019()` yang berfungsi mencari data tertentu dengan cara menelusuri node mulai dari head hingga node terakhir. Program akan membandingkan data pada setiap node dengan data yang dicari. Jika data ditemukan maka method akan mengembalikan nilai `true`, sedangkan jika data tidak ditemukan hingga akhir list maka method akan mengembalikan nilai `false`. Selain itu terdapat method `traversal_3019()` yang digunakan untuk menampilkan seluruh isi linked list secara berurutan. Pada method `main`, dibuat linked list dengan data 14, 21, 13, 30, dan 10, kemudian dilakukan proses pencarian data 30 sehingga program menampilkan bahwa data berhasil ditemukan.

- 3) Run program hingga keluar output seperti berikut, apabila tidak seperti pada gambar, cek kesalahan pada syntax/ kekurangan syntaxnya, dan perbaiki.



```

<terminated> PencarianSLL_2511533019 [Java Appl
Penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu

```

2.1.4 Membuat program keempat (HapusSLL)

- 1) Klik kanan package `pekan5` dan pilih “New”, pilih class. Lalu buat nama “`HapusSLL_2511533019`”.
- 2) Masukkan syntax seperti berikut.

```

1) package Pekan5_2511533019;
2)
3) public class HapusSLL_2511533019 {
4)     //fungsi untuk menghapus head

```

```

5) public static NodeSLL_2511533019
   deleteHead_3019(NodeSLL_2511533019 head_3019) {
6)     //jika SLL kosong
7)     if (head_3019== null);
8)     //pindahkan head ke node berikutnya
9)     head_3019 = head_3019.next_3019;
10)    //return head baru
11)    return head_3019;
12) }
13) //fungsi menghapus node trakhir SLL
14) public static NodeSLL_2511533019
   removeLastNode_3019(NodeSLL_2511533019 head_3019) {
15)    //jika list kosong, return null
16)    if(head_3019 == null) {
17)        return null;
18)    }
19)    //jika list satu node, hapus node dan return
    null
20)    if (head_3019.next_3019== null) {
21)        return null;
22)    }
23)    //temukan node terakhir ke dua
24)    NodeSLL_2511533019 secondLast_3019 =
    head_3019;
25)    while (secondLast_3019.next_3019.next_3019 !=
    null) {
26)        secondLast_3019 =
    secondLast_3019.next_3019;
27)    }
28)    // hapus node terakhir
29)    secondLast_3019.next_3019= null;
30)    return head_3019; }
31)
32) //fungsi menghapus node di posisi tertentu
33) public static NodeSLL_2511533019
   deleteNode_3019(NodeSLL_2511533019 head_3019, int
   position_3019) {
34)     NodeSLL_2511533019 temp_3019=head_3019;
35)     NodeSLL_2511533019 prev_3019=null;
36)    // jika linked list null
37)    if (temp_3019 == null)
38)        return head_3019;
39)    //kasus 1: head dihapus
40)    if (position_3019 == 1) {
41)        head_3019 = temp_3019.next_3019;
42)        return head_3019;
43)    }
44)    // kasus 2: menghapus node di tengah
45)    // telusuri ke node yang dihapus
46)    for (int i = 1; temp_3019 != null && i <
    position_3019; i++) {
47)        prev_3019 = temp_3019;
48)        temp_3019 = temp_3019.next_3019;}
49)    // jika ditemukan, hapus node
50)    if (temp_3019 != null) {
51)        prev_3019.next_3019 = temp_3019.next_3019;
52)    } else {

```

```

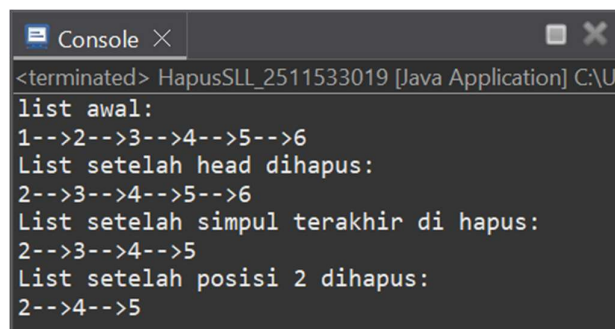
53)         System.out.println ("Data tidak ada");}
54) return head_3019;
55) }
56) //fungsi mencetak SLL
57) public static void printList_3019(NodeSLL_2511533019
    head_3019) {
58)     NodeSLL_2511533019 curr_3019 = head_3019;
59)     while (curr_3019.next_3019 != null) {
60)         System.out.print(curr_3019.data_3019
            + "-->");
61)         curr_3019 = curr_3019.next_3019; }
62)     if (curr_3019.next_3019==null) {
63)         System.out.print(curr_3019.data_3019);}
64)     System.out.println(); }
65)
66) //kelas main
67) public static void main(String[] args) {
68)     // TODO Auto-generated method stub
69)     // buat SLL 1 -> 2 -> 3 -> 4 -> 5-> 6 -> null
70)     NodeSLL_2511533019 head_3019 = new
        NodeSLL_2511533019(1);
71)     head_3019.next_3019=new
        NodeSLL_2511533019(2);
72)     head_3019.next_3019.next_3019=new
        NodeSLL_2511533019(3);
73)     head_3019.next_3019.next_3019.next_3019=new
        NodeSLL_2511533019(4);
74)
        head_3019.next_3019.next_3019.next_3019.next_3019
        =new NodeSLL_2511533019(5);
75)
        head_3019.next_3019.next_3019.next_3019.next_3019.ne
        xt_3019= new NodeSLL_2511533019(6);
76)     // cetak list awal
77)     System.out.println ("list awal: ");
78)     printList_3019(head_3019);
79)     // hapus head
80)     head_3019=deleteHead_3019(head_3019);
81)     System.out.println("List setelah head
        dihapus: ");
82)     printList_3019(head_3019);
83)     //hapus node terakhir
84)     head_3019=removeLastNode_3019(head_3019);
85)     System.out.println ("List setelah simpul
        terakhir di hapus: ");
86)     printList_3019(head_3019);
87)     // Deleting node at position 2
88)     int position_3019 = 2;
89)     head_3019 = deleteNode_3019(head_3019,
        position_3019);
90)     // Print list after deletion
91)     System.out.println("List setelah posisi 2
        dihapus: ");
92)     printList_3019(head_3019) ;
93)
94) }
95)

```

```
96) }
```

Program terakhir yaitu HapusSLL_2511533019 yang digunakan untuk melakukan operasi penghapusan node pada Single Linked List. Method deleteHead_3019() digunakan untuk menghapus node pertama dengan cara memindahkan head ke node berikutnya. Method removeLastNode_3019() digunakan untuk menghapus node terakhir dengan mencari node sebelum node terakhir kemudian mengubah pointer-nya menjadi null. Selain itu, method deleteNode_3019() digunakan untuk menghapus node pada posisi tertentu dengan cara menghubungkan node sebelumnya ke node setelah node yang dihapus. Pada program ini dibuat linked list dengan data 1, 2, 3, 4, 5, dan 6. Setelah dilakukan penghapusan head, penghapusan node terakhir, dan penghapusan node pada posisi tertentu, linked list berhasil diperbarui sesuai operasi yang dilakukan.

- 3) Run program hingga mengeluarkan output seperti berikut.



```
Console X
<terminated> HapusSLL_2511533019 [Java Application] C:\U
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Setelah di run, pada awal ditampilkan linked list berisi data 1-->2-->3-->4-->5-->6 sebagai list awal. Selanjutnya program menjalankan method deleteHead_3019() untuk menghapus node pertama atau head sehingga data 1 terhapus dan linked list berubah menjadi 2-->3-->4-->5-->6. Setelah itu program menjalankan method removeLastNode_3019() untuk menghapus node terakhir dengan cara mencari node sebelum node terakhir lalu mengubah pointer-nya menjadi null, sehingga data 6 terhapus dan linked list menjadi 2-->3-->4-->5. Kemudian program menjalankan method deleteNode_3019() untuk menghapus node

pada posisi ke-2, yaitu data 3, sehingga node sebelumnya langsung terhubung ke node setelahnya dan menghasilkan linked list akhir 2-->4-->5. Output tersebut menunjukkan bahwa operasi penghapusan pada bagian depan, belakang, dan posisi tertentu pada Single Linked List berhasil dilakukan dengan baik.

2.2 Tugas Praktikum

2.2.1 Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO : First In, First Out).

2.2.2 Kelas ADT

- 1) Klik kanan package pekan5 dan pilih “New”, pilih class. Lalu buat nama “**Pasien_2511533019**”.
- 2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1) package Pekan5_2511533019;
2)
3) public class Pasien_2511533019 {
4)     String NamaPasien_3019;
5)     String keluhan_3019;
6)     int noAntrian_3019;
7)     //pointer ke node pasien berikutnya
8)     Pasien_2511533019 next_3019;
9)     //konstruktor menginisialisasi node
    dengan data
10)    public Pasien_2511533019(String
    NamaPasien_3019, String keluhan_3019, int
    noAntrian_3019) {
11)        this>NamaPasien_3019=
        NamaPasien_3019;
12)        this.keluhan_3019=keluhan_3019;
```

```

13)         this.noAntrian_3019=
noAntrian_3019;
14)         this.next_3019 = next_3019;
15)     }
16)     //getter
17)     public String getNamaPasien_3019() {
18)         return>NamaPasien_3019;
19)     }
20)     public String getkeluhan_3019() {
21)         return>keluhan_3019;
22)     }
23)     public int getnoAntrian_3019() {
24)         return>noAntrian_3019;
25)     }
26)     public>Pasien_2511533019 getNext_3019()
{
27)         return>next_3019;
28)     }
29)     //setter
30)     public void setNamaPasien_3019(String
>NamaPasien_3019) {
31)         this>NamaPasien_3019=
>NamaPasien_3019;
32)     }
33)
34)     public void setkeluhan_3019(String
>keluhan_3019) {
35)         this.keluhan_3019= keluhan_3019;
36)     }
37)
38)     public void setnoAntrian_3019(int
>noAntrian_3019) {
39)         this.noAntrian_3019=noAntrian_3019;
40)     }
41)
42)     public void
>setnext_3019(Pasien_2511533019 next_3019) {
43)         this.next_3019= next_3019;
44)     }
45) }

```

Program ini dibuat untuk menyimpan data pasien pada struktur data Single Linked List (SLL) dalam simulasi antrian rumah sakit. Pada class ini terdapat beberapa atribut yaitu>NamaPasien_3019 untuk menyimpan nama pasien, keluhan_3019 untuk menyimpan keluhan/penyakin yang dimiliki pasien, noAntrian_3019 untuk menyimpan nomor antrian pasien, serta next_3019 yang berfungsi sebagai pointer untuk menghubungkan node pasien dengan node berikutnya.

Konstruktor pada class ini digunakan untuk menginisialisasi data pasien ketika object dibuat, sehingga setiap node langsung memiliki data nama pasien, keluhan, dan nomor antrian.

Selain itu, class ini juga memiliki beberapa method getter seperti `getNamaPasien_3019()`, `getkeluhan_3019()`, `getnoAntrian_3019()`, dan `getnext_3019()` yang digunakan untuk mengambil nilai atribut pada object pasien.

Terdapat pula method setter seperti `setNamaPasien_3019()`, `setkeluhan_3019()`, `setnoAntrian_3019()`, dan `setnext_3019()` yang digunakan untuk mengubah nilai atribut pasien maupun menghubungkan node ke node berikutnya. Dengan adanya class ini, data pasien dapat disimpan dan dikelola secara dinamis menggunakan konsep Single Linked List.

2.2.3 Kelas Driver

- 1) Klik kanan package `pekan5` dan pilih “New”, pilih class. Lalu buat nama “**RumahSakit_2511533019**”.
- 2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1) package Pekan5_2511533019;
2)
3) public class RumahSakit_2511533019 {
4)     public static Pasien_2511533019
insertPasien_3019(Pasien_2511533019 head_3019,
String NamaPasien_3019, String keluhan_3019, int
noAntrian_3019) {
5)         Pasien_2511533019 pasienBaru_3019 = new
Pasien_2511533019(NamaPasien_3019, keluhan_3019,
noAntrian_3019);
6)         //jika list kosong maka node jadi head
7)         if(head_3019 == null) {
8)             return pasienBaru_3019;
9)         }
10)        // menuju node terakhir
11)        Pasien_2511533019 last_3019 = head_3019;
12)        while (last_3019.next_3019 != null) {
13)            last_3019 = last_3019.next_3019;
14)        }
15)        // hubungkan node baru
16)        last_3019.next_3019 = pasienBaru_3019;
17)        return head_3019;
18)    }
```

```

19)      // method tampil antrian
20)      public static void
21)      display_3019(Pasien_2511533019 head_3019) {
22)          Pasien_2511533019 curr_3019 =
23)          head_3019;
24)          int noAntrian_3019 = 1;
25)          while (curr_3019 != null) {
26)              System.out.println(noAntrian_3019 + ". " +
27)              curr_3019>NamaPasien_3019 + " " +
28)              curr_3019.keluhan_3019);
29)              curr_3019 = curr_3019.next_3019;
30)              noAntrian_3019++;
31)          }
32)      }
33)
34)      // method panggil pasien(delete head)
35)      public static Pasien_2511533019
36)      panggilPasien_3019(Pasien_2511533019 head_3019) {
37)          if (head_3019 == null) {
38)              System.out.println("Antrian
39)              kosong");
40)              return null;
41)          }
42)          System.out.println("Pasien dipanggil :
43)          "+ head_3019>NamaPasien_3019);
44)          head_3019 = head_3019.next_3019;
45)          return head_3019;
46)      }
47)      // method cari pasien
48)      public static boolean
49)      cariPasien_3019(Pasien_2511533019 head_3019,String
50)      namaCari_3019) {
51)          Pasien_2511533019 curr_3019 =
52)          head_3019;
53)          while (curr_3019 != null) {
54)              if
55)              (curr_3019>NamaPasien_3019.equalsIgnoreCase(namaCar
56)              i_3019)) {
57)                  System.out.println("Pasien ditemukan");
58)                  System.out.println(curr_3019.noAntrian_3019 +
59)                  " " + curr_3019>NamaPasien_3019 + " " +
60)                  curr_3019.keluhan_3019);
61)                  return true;
62)              }
63)              curr_3019 =
64)              curr_3019.next_3019;
65)          }
66)      }

```

```

51)         System.out.println("Pasien tidak
           ditemukan");
52)         return false;
53)     }
54)     // method cek status antrian
55)     public static void
           cekStatus_3019(Pasien_2511533019 head_3019) {
56)         if (head_3019 == null) {
57)             System.out.println("Antrian
           kosong");
58)             return;
59)         }
60)         int jumlah_3019 = 0;
61)         Pasien_2511533019 curr_3019 =
           head_3019;
62)         while (curr_3019 != null) {
63)             jumlah_3019++;
64)             curr_3019 =
           curr_3019.next_3019;
65)         }
66)         System.out.println("Jumlah
           pasien dalam antrian : "+ jumlah_3019);
67)         System.out.println("Pasien
           terdepan : "+ head_3019>NamaPasien_3019);
68)     }
69)
70)     public static void main(String[] args) {
71)         Pasien_2511533019 head_3019 = null;
72)         System.out.println("=== Antrian Rumah
           Sakit NIM: 2511533019 ===");
73)         System.out.println("1.Daftarkan Pasien
           (Insert)");
74)         System.out.println("2.Panggil Pasien
           (Delete Head)");
75)         System.out.println("3.Tampilkan Antrian
           (Display)");
76)         System.out.println("4.Cari Pasien
           (Search)");
77)         System.out.println("5.Cek Status
           Antrian");
78)         System.out.println("6.Keluar");
79)         //pasien pertama
80)         System.out.println("\nPilihan: 1");
81)         System.out.println("Masukkan Nama
           Pasien : Budi Santoso");
82)         System.out.println("Masukkan Keluhan
           : Demam");
83)         head_3019 = new Pasien_2511533019("Budi
           Santoso", "Demam",1);
84)         System.out.println("Pasien berhasil
           didaftarkan! No Antrian: 1");

```

```

85)
86)         // pasien kedua
87)         System.out.println("\nPilihan: 1");
88)         System.out.println("Masukkan Nama
Pasien : Sinta");
89)         System.out.println("Masukkan Keluhan
: Influenza");
90)         head_3019.next_3019 = new
Pasien_2511533019("Sinta", "Influenza", 2);
91)         System.out.println("Pasien berhasil
didaftarkan! No Antrian: 2");
92)
93)         // pasien ketiga
94)         System.out.println("\nPilihan: 1");
95)         System.out.println("Masukkan Nama
Pasien : Andi");
96)         System.out.println("Masukkan Keluhan
: Gastritis");
97)         head_3019.next_3019.next_3019 = new
Pasien_2511533019("Andi", "Gastritis", 3);
98)         System.out.println("Pasien berhasil
didaftarkan! No Antrian: 3");
99)
100)        // tampil antrian
101)        System.out.println("\nPilihan:
3");
102)        System.out.println("Daftar
Antrian:");
103)        display_3019(head_3019);
104)
105)        // cari pasien
106)        System.out.println("\nPilihan:
4");
107)        System.out.println("Cari Pasien
: Sinta");
108)        cariPasien_3019(head_3019,
"Sinta");
109)
110)        // cek status
111)        System.out.println("\nPilihan:
5");
112)        cekStatus_3019(head_3019);
113)
114)        // panggil pasien
115)        System.out.println("\nPilihan:
2");
116)        head_3019 =
panggilPasien_3019(head_3019);
117)
118)        // tampil antrian lagi

```

```
119)                System.out.println("\nPilihan:
    3");
120)                System.out.println("Daftar
    Antrian:");
121)                display_3019(head_3019);
122)                System.out.println("\nPilihan:
    6");
123)                System.out.println("Program
    selesai");
124)                }
125)                }
```

3) Run program :

```
Console X
<terminated> RumahSakit_2511533019 [Java Application] C:\Users\Filzi Jelila\.p2\poc
=== Antrian Rumah Sakit NIM: 2511533019 ===
1.Daftarkan Pasien (Insert)
2.Panggil Pasien (Delete Head)
3.Tampilkan Antrian (Display)
4.Cari Pasien (Search)
5.Cek Status Antrian
6.Keluar

Pilihan: 1
Masukkan Nama Pasien : Budi Santoso
Masukkan Keluhan : Demam
Pasien berhasil didaftarkan! No Antrian: 1

Pilihan: 1
Masukkan Nama Pasien : Sinta
Masukkan Keluhan : Influenza
Pasien berhasil didaftarkan! No Antrian: 2

Pilihan: 1
Masukkan Nama Pasien : Andi
Masukkan Keluhan : Gastritis
Pasien berhasil didaftarkan! No Antrian: 3

Pilihan: 3
Daftar Antrian:
1. Budi Santoso Demam
2. Sinta Influenza
3. Andi Gastritis

Pilihan: 4
Cari Pasien : Sinta
Pasien ditemukan
2 Sinta Influenza

Pilihan: 5
Jumlah pasien dalam antrian : 3
Pasien terdepan : Budi Santoso

Pilihan: 2
Pasien dipanggil : Budi Santoso

Pilihan: 3
Daftar Antrian:
1. Sinta Influenza
2. Andi Gastritis

Pilihan: 6
Program selesai
```

BAB III

PENUTUP

3.1 Kesimpulan

Dari seluruh praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Single Linked List (SLL) merupakan struktur data dinamis yang terdiri dari node-node yang saling terhubung melalui pointer. Pada praktikum ini berhasil diimplementasikan berbagai operasi dasar pada Single Linked List seperti traversal atau penelusuran data, pencarian data, penambahan node di depan, belakang, maupun posisi tertentu, serta penghapusan node pada bagian awal, akhir, dan posisi tertentu. Melalui implementasi tersebut, mahasiswa dapat memahami bagaimana data disimpan dan dikelola secara dinamis tanpa harus menggunakan ukuran memori tetap seperti array.

Selain itu, praktikum ini juga menunjukkan bahwa Single Linked List dapat diterapkan dalam berbagai kasus sederhana di kehidupan nyata, salah satunya pada simulasi antrian rumah sakit. Pada program tersebut, konsep linked list digunakan untuk mengelola data pasien secara terurut mulai dari proses pendaftaran pasien, pemanggilan pasien, pencarian data pasien, menampilkan daftar antrian, hingga pengecekan status antrian. Implementasi ini membantu mahasiswa memahami hubungan antara konsep struktur data dengan penerapannya dalam pemrograman berbasis objek menggunakan bahasa Java.

Secara keseluruhan, praktikum ini membantu meningkatkan pemahaman mengenai konsep dasar Single Linked List, melatih kemampuan implementasi struktur data dalam bahasa pemrograman Java, serta mengembangkan logika berpikir mahasiswa dalam menyelesaikan permasalahan pengolahan data secara dinamis dan terstruktur.

DAFTAR PUSTAKA

- [1] Wahyudi, “*Linked List*” PowerPoint slides, Praktikum Struktur Data, Universitas Andalas, Padang, 2025.
- [2] Oracle, “LinkedList (Java Platform SE 8),” [Online]. Available:
<https://docs.oracle.com/javase/8/docs/api/java/util/LinkedList.html>